CITS4404 — Deliverable 1

Nature-Inspired Algorithm Synopses

Team Number: 4

1. Bomin Liao 24331475 2. Weishan Li 24746589
3. Ziqi Meng 24645175 4. Chenxiao Jiang 24438869

Algorithms Covered:

1. Grey Wolf Optimizer (GWO)
2. Simulated Annealing (SA)
3. Differential Evolution (DE)
4. Particle Swarm Optimization (PSO)

Synopsis 1: Grey Wolf Optimizer (GWO)

S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey Wolf Optimizer," Advances in Engineering Software, vol. 69, pp. 46–61, 2014.

Category: Swarm Intelligence
Author: Bomin Liao

Q1. What problem with existing algorithms is GWO attempting to solve?

Despite the widespread adoption of Swarm Intelligence (SI) [1] in optimization, no prior algorithm had leveraged the social hierarchy of grey wolves to direct the search process. GWO draws on the pack's leadership structure and cooperative hunting behavior to guide candidate solutions, using the top three solutions simultaneously to achieve a more principled balance between exploration and exploitation.

Q2. Why, or in what respect, have previous attempts failed?

Evolutionary algorithms such as GA [2] and DE [3] discard population information between generations, effectively wasting accumulated search knowledge. More broadly, existing metaheuristics struggled to reliably balance exploration and exploitation — a fundamental tension in optimization. Within the SI category specifically, no algorithm had exploited a social dominance hierarchy as an organizing principle for search, leaving this mechanism unexplored.

Q3. What is the new idea presented in this paper?

GWO [4] encodes candidate solutions into four ranks — α, β, δ, and ω — mirroring the grey wolf pack hierarchy. The three highest-ranked solutions (α, β, δ) collectively steer the positions of all other wolves, preventing over-reliance on a single best solution. A linearly decaying parameter A controls the transition between behaviors: when $|A| \geq 1$, wolves diverge to explore; when $|A| < 1$, they converge to exploit. A randomized coefficient C further perturbs the influence of the prey position, sustaining diversity throughout the search.

Q4. How is the new approach demonstrated?

GWO is benchmarked on 29 standard test functions spanning four categories: unimodal (F1–F7, exploitation), multimodal (F8–F13, exploration), fixed-dimension multimodal (F14–F23, local optima avoidance), and composite (F24–F29, combined). Each function is run 30 times, with mean and standard deviation reported. Comparisons are drawn against PSO [5], GSA [6], DE [3], EP, and ES. The algorithm is further evaluated on three constrained engineering design problems — tension/compression spring, welded beam, and pressure vessel — and a real optical engineering problem involving photonic crystal waveguide optimization.

Q5. What are the results or outcomes and how are they validated?

On unimodal functions, GWO outperforms all competitors on F1, F2, and F7, demonstrating strong exploitation. On multimodal and fixed-dimension functions, it surpasses PSO and GSA on the majority of cases. For composite functions, GWO leads on half and remains competitive on the rest. In the engineering problems, GWO produces the lightest spring, lowest-cost welded beam, and lowest-cost pressure vessel — outperforming PSO, GA, DE, and ES across the board. In the optical engineering application, GWO achieves a 93% improvement in bandwidth and 65% improvement in NDBP over the previous state of the art [4].

Q6. What is your assessment of the conclusions?

The conclusions are well-supported by a thorough experimental design spanning benchmark functions, constrained engineering problems, and a real application. That said, the comparison set (EP, ES, FEP) is largely dated, and the paper does not test GWO in high-dimensional settings beyond 30 variables. For this project, GWO's compact parameter set and adaptive exploration–exploitation balance make it well-suited to optimizing trading bot parameters such as moving average window sizes, where the fitness landscape is continuous and the objective function (e.g., return on investment) is inexpensive to evaluate.

Synopsis 2: Simulated Annealing (SA)

S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," Science, vol. 220, no. 4598, pp. 671–680, May 1983.

Category: Single-State Stochastic

Author: Weishan Li

Q1. What problem with existing algorithms is SA attempting to solve?

SA escapes local optima by accepting worse outcomes with a certain probability to obtain the true optimal result. For NP-complete problems, exact solution methods require computing effort that

grows exponentially with problem size, making them practically infeasible [7]. SA was proposed as a practical heuristic that can escape local optima and address the problem of slow convergence by probabilistically accepting worse moves rather than greedily descending, effectively balancing exploration and exploitation [8].

Q2. Why, or in what respect, have previous attempts failed?

Iterative improvement methods, the dominant heuristic paradigm at the time, accept only cost-reducing changes. This is mathematically equivalent to quenching a physical system to absolute zero, that result is a metastable (locally optimal) configuration whose quality depends heavily on the starting point [7]. Running multiple restarts from random initializations mitigates this somewhat but offers no convergence guarantee and is computationally wasteful.

Q3. What is the new idea presented in this paper?

SA adapts the Metropolis algorithm from statistical mechanics: a candidate move with cost increase ΔE is accepted with probability $\exp(-\Delta E/T)$, where T is a control parameter analogous to temperature. The simulated annealing process mirrors the physical cooling process in metallurgy. At high T, nearly all moves are accepted, enabling broad exploration; as T is gradually lowered according to a cooling schedule, the algorithm increasingly favors improvements, shifting into exploitation mode. This temperature-driven mechanism allows SA to escape local optima while still converging [9].

Q4. How is the new approach demonstrated?

The algorithm is applied to three physical design problems from IBM computer engineering: (1) partitioning 98 chips from an IBM 370 processor across two chip carriers to minimize inter-chip connections; (2) chip placement to minimize wire length and congestion; and (3) global wire routing to minimize peak wire density. A 400-city Travelling Salesman Problem is also used to assess broader applicability. Results at each stage are compared with random assignment and rapid quenching (greedy descent at T = 0) [7].

Q5. What are the results or outcomes and how are they validated?

For chip partitioning, SA yields a solution requiring approximately 271 and 183 pins per carrier, compared to roughly 700 pins under rapid quenching — a substantial reduction. Chip placement after annealing shows peak wire-crossing histograms roughly 30% flatter than the original design, with total wire length reduced by around 10%, achieved in 250,000 exchanges (approximately 12 minutes on the hardware of the time). Wire routing with SA improves on random assignment by 57%. TSP results show coherent local structure emerging as temperature falls, confirming the cooling mechanism works as intended [7].

Q6. What is your assessment of the conclusions?

The results are convincing within their domain, and the analogy to physical annealing provides a theoretically grounded motivation. However, the evaluation lacks statistical rigour: no variance across multiple runs is reported, and there is no comparison against other heuristics of similar complexity. The cooling schedule is chosen empirically with no general guidelines, which limits reproducibility. SA requires a large number of function evaluations to converge, particularly at low temperatures [10]. In this project, SA's single-solution nature means lower memory overhead than population-based methods, making it a useful baseline against which PSO, DE, and GWO can be benchmarked.

Synopsis 3: Differential Evolution (DE)

R. Storn and K. Price, "Differential evolution — A simple and efficient heuristic for global optimization over continuous spaces," Journal of Global Optimization, vol. 11, no. 4, pp. 341–359, 1997.

Category: Evolutionary Algorithm

Author: Ziqi Meng

Q1. What problem with existing algorithms is DE attempting to solve?

Many real-world continuous optimization problems — including trading strategy parameter tuning — involve non-smooth, non-differentiable, or noisy objective functions that gradient-based methods cannot handle [3]. DE was introduced to provide a reliable, derivative-free approach to global optimization over continuous search spaces, with minimal algorithmic complexity.

Q2. Why, or in what respect, have previous attempts failed?

Classical numerical methods require derivative information and are prone to local optima. Earlier evolutionary approaches such as Genetic Algorithms [2], while derivative-free, impose significant overhead through encoding schemes, crossover operators, and multi-parameter tuning, making them cumbersome in practice. These drawbacks motivated the design of a simpler evolutionary framework that retains global search capability without unnecessary complexity [3].

Q3. What is the new idea presented in this paper?

DE's central contribution is a vector-difference mutation operator: a trial vector is constructed as $v = x_1 + F \cdot (x_2 - x_3)$, where x_1, x_2, x_3 are distinct randomly selected population members and $F \in (0, 2]$ is a scaling factor. This scheme adapts step sizes to the current population spread, allowing DE to self-tune its exploration radius without external annealing schedules or complex operators. A binomial crossover then blends the trial vector with the target individual, and the better of the two is retained [3].

Q4. How is the new approach demonstrated?

The algorithm iterates through four stages per generation: (1) initialization of a random population; (2) mutation via the vector-difference formula; (3) crossover between mutant and target vectors; (4) greedy selection retaining the fitter individual. Storn and Price evaluate DE on a suite of standard continuous benchmark functions and compare it against other evolutionary optimizers, reporting convergence speed and final solution accuracy [3].

Q5. What are the results or outcomes and how are they validated?

Across standard benchmarks, DE consistently achieves faster convergence and equal or better accuracy than contemporary algorithms including GA and early versions of PSO, particularly on continuous, separable problems [3], [11]. Performance is measured by the number of function evaluations required to reach a target accuracy and by final solution quality, providing a clear and reproducible validation framework.

Q6. What is your assessment of the conclusions?

The performance claims are well-supported and DE has since become one of the most widely

used continuous optimizers, validating the original conclusions [11]. The two key parameters — scaling factor F and crossover rate CR — are relatively intuitive to set, and the algorithm's behavior is predictable. For this project, DE is a strong candidate because the trading bot's parameter space is continuous and bounded, which is precisely the setting DE was designed for.

Synopsis 4: Particle Swarm Optimization (PSO)

J. Kennedy and R. Eberhart, "Particle swarm optimization," Proc. IEEE Int. Conf. Neural Networks, 1995.

Category: Swarm Intelligence

Author: Chenxiao Jiang

Q1. What problem with existing algorithms is PSO attempting to solve?

Kennedy and Eberhart observed that Genetic Algorithms, while capable of global search, carry substantial overhead from crossover, mutation, and selection operators, and are sensitive to hyperparameter choices [5]. Motivated by the collective foraging behavior of bird flocks and fish schools, they sought a population-based optimizer driven purely by social information sharing — one that could match GA's search power with far less implementation complexity.

Q2. Why, or in what respect, have previous attempts failed?

GA's operators introduce computational overhead and require domain-specific encoding decisions. Sensitivity to population size, crossover rate, and mutation rate makes GA difficult to apply without prior expertise. There was a clear need for a population-based algorithm with fewer control parameters and a more transparent update mechanism.

Q3. What is the new idea presented in this paper?

PSO maintains a swarm of particles, each carrying its current position x, velocity v, personal best position pbest, and the swarm's global best gbest. At each iteration, velocity is updated as: v_id $\leftarrow$ v_id + $2r_1$(pbest_id − x_id) + $2r_2$(gbest_d − x_id), where r_1, $r_2 \in [0,1]$ are random scalars. Position is then updated by x $\leftarrow$ x + v. The cognitive term (pbest) preserves individual experience, while the social term (gbest) enables collective convergence — all without genetic operators.

Q4. How is the new approach demonstrated?

PSO is evaluated on two tasks: training a feedforward neural network on the XOR problem and the Fisher Iris dataset, and optimizing Schaffer's f6 function — a multimodal benchmark commonly used for GA evaluation. The paper traces the algorithm's development from Reynolds' flocking simulations [12] and Millonas' swarm intelligence principles [13] through to the final velocity–position update rule. Reproducibility is somewhat limited by the absence of fixed random seeds.

Q5. What are the results or outcomes and how are they validated?

On the XOR task, PSO matches standard backpropagation; on EEG classification it achieves 92% accuracy versus backpropagation's 89%. On Schaffer's f6, PSO locates the global optimum on every trial with a function evaluation count comparable to elementary GA. These results indicate that PSO is competitive with backpropagation in generalization metrics and remains competitive with GA on benchmark optimization, while offering much lower implementation complexity.

Q6. What is your assessment of the conclusions?

The original conclusions hold up well: PSO is simple, effective, and easy to implement. The main limitation acknowledged only in subsequent literature is premature convergence on multimodal problems — particles collapse towards a suboptimal gbest before the space is adequately explored. For this project, PSO's straightforward velocity–position update makes it easy to implement from scratch, and its population-based nature suits the multi-dimensional continuous parameter space of the trading bot optimization task.

Conclusions

The four algorithms selected for this review span two generations and three distinct search paradigms. SA (1983) represents the earliest class of stochastic single-solution methods, while PSO (1995) , DE (1997) , and GWO (2014) are population-based approaches. The timeline is shown in Figure 1. There is a clear chronological progression, with newer algorithms generally reporting stronger benchmark results — though no algorithm is universally superior across all problem types [15].

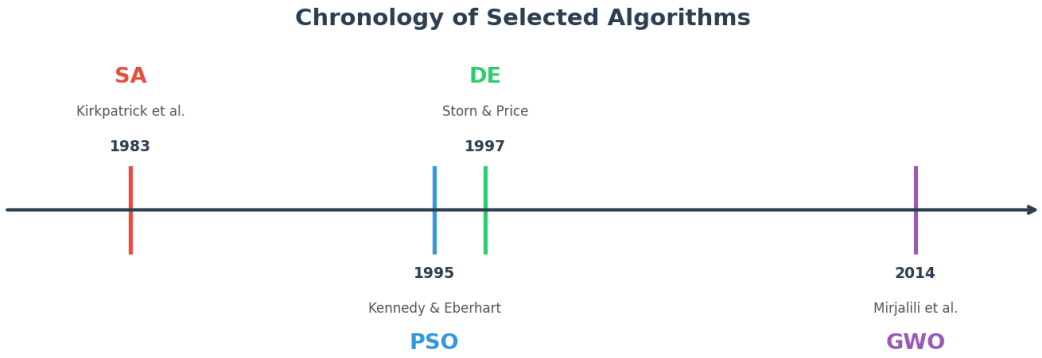

Figure 1. Chronology of the four selected algorithms — year of original publication for each method.

In terms of structural diversity, SA operates on a single candidate solution, accepting worse moves probabilistically to escape local optima. The three population-based methods maintain a set of candidate solutions that interact and evolve over iterations. Within this group, DE is classified as an evolutionary algorithm, using vector-difference mutation; PSO and GWO are both Swarm Intelligence methods, but differ in their update mechanisms — PSO relies on velocity–position dynamics informed by personal and global bests, while GWO uses a social hierarchy (α, β, δ) to collectively estimate the prey location. The taxonomy of these relationships is illustrated in Figure 2.

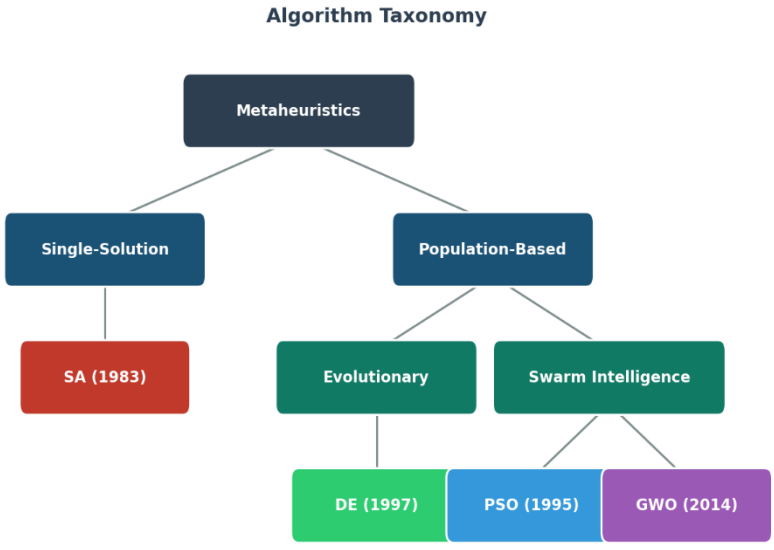

Figure 2. Algorithm taxonomy — classification of the four algorithms into Single-Solution and Population-Based categories, following the framework described in [1].

All four algorithms share the same high-level objective: minimize a fitness function to locate the global optimum. In GWO the fittest solution is designated α; in PSO it is the global best. Despite this shared goal, their convergence profiles differ substantially. SA converges slowly and steadily, with theoretical guarantees under idealized cooling schedules but requiring a large number of evaluations in practice [10]. PSO converges rapidly in the early stages but is prone to premature stagnation on multimodal landscapes [14]. DE exhibits robust global search behavior but is sensitive to the scaling factor F and crossover rate CR [3]. GWO achieves a well-balanced trade-off between exploration and exploitation through its adaptive A parameter, though it can sacrifice final solution accuracy on high-precision problems [15]. These differences in convergence behavior are illustrated schematically in Figure 3.

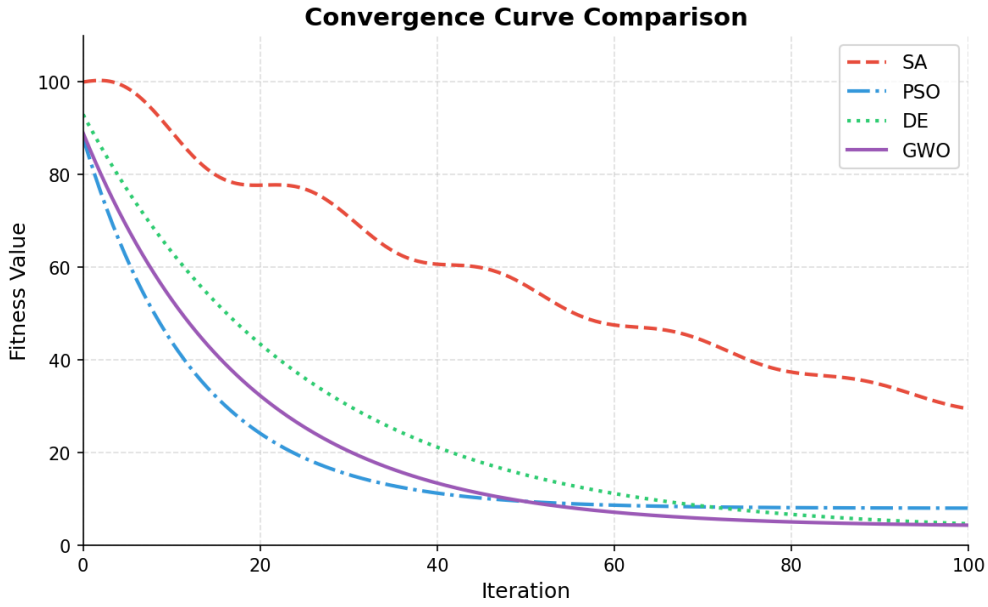

Figure 3. Illustrative convergence behavior of the four algorithms (schematic). Based on reported characteristics in GWO, SA, DE, and PSO.

Based on this analysis, DE and PSO are selected for further implementation in Part 2: DE for its proven strength in continuous parameter optimization, and PSO for its fast convergence and straightforward implementation, allowing a direct comparison between an evolutionary and a swarm-based approach. A summary comparison of the four methods is shown in Table 1 below.

Table 1. Summary comparison of the four selected algorithms.

Algorithm	Type	Parameters	Strength	Weakness
SA	Single solution	Few	Escapes local optima	Slow convergence
PSO	Swarm	Few	Fast convergence	Premature convergence
DE	Evolutionary	Few	Strong for continuous	Parameter sensitivity
GWO	Swarm	Moderate	Balanced search	Less precise at high dimension

References

[1] E. Bonabeau, M. Dorigo, and G. Theraulaz, Swarm Intelligence: From Natural to Artificial Systems. New York: Oxford University Press, 1999.

[2] J. H. Holland, “Building blocks, cohort genetic algorithms, and hyperplane-defined functions,” *Evolutionary Computation*, vol. 8, no. 4, pp. 373–391, Dec. 2000.

[3] R. Storn and K. Price, "Differential evolution — A simple and efficient heuristic for global optimization over continuous spaces," Journal of Global Optimization, vol. 11, no. 4, pp. 341–359, 1997.

[4] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey Wolf Optimizer," Advances in Engineering Software, vol. 69, pp. 46–61, 2014.

[5] J. Kennedy and R. Eberhart, "Particle swarm optimization," in Proc. ICNN'95 — Int. Conf. Neural Networks, Perth, WA, Australia, 1995, vol. 4, pp. 1942–1948.

[6] E. Rashedi, H. Nezamabadi-Pour, and S. Saryazdi, "GSA: A gravitational search algorithm," Information Sciences, vol. 179, no. 13, pp. 2232–2248, 2009.

[7] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," Science, vol. 220, no. 4598, pp. 671–680, May 1983.

[8] F. Sajjad, M. Rashid, A. Zafar, K. Zafar, B. Fida, A. Arshad, S. Riaz, A. K. Dutta, and J. J. P. C. Rodrigues, “An efficient hybrid approach for optimization using simulated annealing and grasshopper algorithm for IoT applications,” *Research*, vol. 3, Art. no. 7, Jul. 2023.

[9] O. Kuznetsov, “Simulated Annealing,” in *Intelligent Systems: From Theory to Applications*, Cognitive Technologies. Cham, Switzerland: Springer, 2025, pp. 233–246. [Online]. Available: https://doi.org/10.1007/978-3-032-00044-6_11

[10] A. T. Kalai and S. Vempala, “Simulated annealing for convex optimization,” Mathematics of Operations Research, vol. 31, no. 2, pp. 253–266, May 2006, doi: 10.1287/moor.1060.0194.

[11] T. Eltaeib and A. Mahmood, "Differential evolution: A survey and analysis," Applied Sciences, vol. 8, no. 10, p. 1945, 2018.

[12] C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model," Computer Graphics, vol. 21, no. 4, pp. 25–34, 1987.

[13] M. M. Millonas, "Swarms, phase transitions, and collective intelligence," in Artificial Life III, C. G. Langton, Ed. Reading, MA: Addison Wesley, 1994.

[14] M. S. Almufti and A. B. Sallow, “Benchmarking metaheuristics: Comparative analysis of PSO, GWO, and ESNS on complex optimization landscapes,” *AVE Trends in Intelligent Computing Systems*, vol. 2, no. 2, pp. 62–76, Jun. 2025.

[15] F. Pace, A. Raftogianni, and A. Godio, “A comparative analysis of three computational-intelligence metaheuristic methods for the optimization of TDEM data,” *Pure and Applied Geophysics*, vol. 179, no. 10, pp. 3727–3749, Oct. 2022.